S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR.

Practical No. 4

Aim: Make use of A* Algorithm to find out goal state (Shortest path cost) in the given Graph. Make use of Breadth First Search algorithm to find the goal state in the above graph and compare the performance of both algorithms in terms of time taken.

Name of Student: _____

Roll No.: CS23___

Semester/Year: V/III

Academic Session: 2025-2026

Date of Performance:_____

Date of Submission:_____

AIM: Make use of A* algorithm to find out goal state in the given problem scenario of Practical No.2.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- To be able to understand the concept of search algorithms.
- To be able to implement the concept of A* algorithm.

THEORY:

Many traditional search algorithms are used in AI applications.

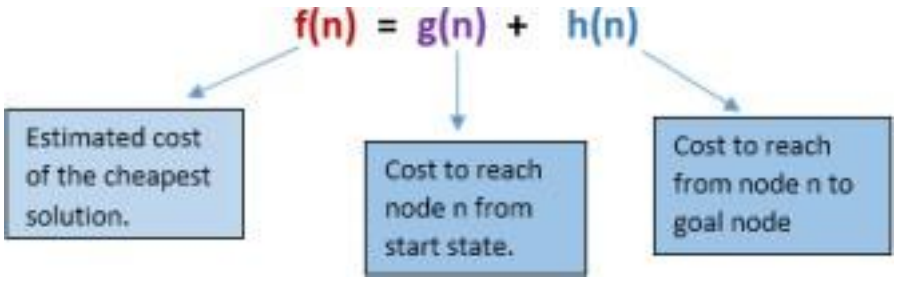
The Search Algorithms Are Classified in Two Types:

Uninformed Search: Also called blind, exhaustive or brute-force search, it uses no information about the problem to guide the search and therefore may not be very efficient. **Informed Search:** Also called heuristic or intelligent search, this uses information about the problem to guide the search—usually guesses the distance to a goal state and is therefore efficient, but the search may not be always possible.

A* Search Algorithm:

A* search is the most commonly known form of best-first search. It uses heuristic function h(n), and cost to reach the node n from the start state. It has combined features of Uniform Cost Search and greedy best-first search, by which it solve the problem efficiently. A* search algorithm finds the shortest path through the search space using the heuristic function. This search algorithm expands less search tree and provides optimal result faster. A* algorithm is similar to UCS except that it uses g(n)+h(n) instead of g(n).

In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as following, and this sum is called as a **fitness number**.



Algorithm of A* Search:

Step1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function (g+h), if node n is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.

Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest g(n') value.

Step 6: Return to **Step 2**.

Advantages:

- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

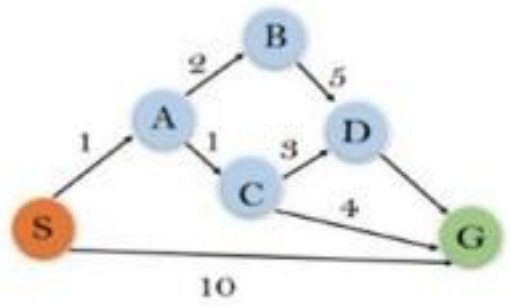
Disadvantages:

- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

Example:

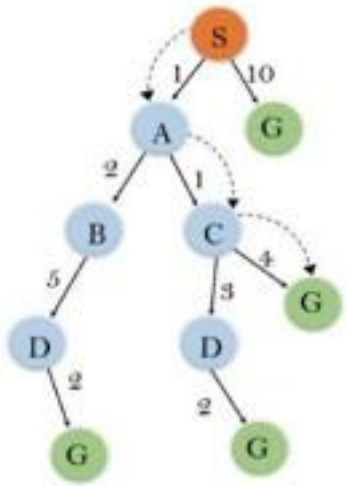
Consider the graph for the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the f(n) of each state using the formula f(n)= g(n) + h(n), where g(n) is the cost to reach any node from start state.

Here we will use OPEN and CLOSED list.



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

Solution:



PSEUDOCODE:

```
Create a node containing the goal state node_goal
Create a node containing the start state node_start
Put node_start on the open list
while the OPEN list is not empty
{
Get the node off the open list with the lowest f and call it node_current
if node_current is the same state as node_goal we have found the solution; break from the while loop
   Generate each state node_successor that can come after node_current
   for each node_successor of node_current
   {
      Set the cost of node_successor to be the cost of node_current plus the cost to get to node_successor from node_current
      find node_successor on the OPEN list
      if node_successor is on the OPEN list but the existing one is as good or better then discard this successor and continue
      if node_successor is on the CLOSED list but the existing one is as good or better then discard this successor and continue
      Remove occurrences of node successor from OPEN and CLOSED
      Set the parent of node_successor to node_current
      Set h to be the estimated distance to node_goal (Using the heuristic function)
      Add node_successor to the OPEN list
   }
   Add node_current to the CLOSED list
```

PROGRAM CODE:

```
from collections import deque
import heapq
g = {'A':[('B',1),('C',4)],
     'B':[('D',2)],
     'C':[('G',3)],
     'D':[('G',1)],
     'G':[]}
h = {'A':4,'B':2,'C':2,'D':1,'G':0}
def astar(s,g1):
    pq=[(0,s,0,[s])]
    while pq:
        f,n,c,p=heapq.heappop(pq)
        if n==g1: return p,c
        for nb,w in g[n]:
            heapq.heappush(pq,(c+w+h[nb],nb,c+w,p+[nb]))
def bfs(s,g1):
    q=deque([(s,[s])])
    while q:
        n,p=q.popleft()
        if n==g1: return p
```

```
        for nb,w in g[n]:
            q.append((nb,p+[nb]))
p1,c=astar('A','G')
p2=bfs('A','G')
print("A*:", *p1, "Cost:", c)
print("BFS:", *p2)
```

OUTPUT:

A Algorithm.py

```python
from collections import deque
import heapq
g = {'A':[('B',1),('C',4)],
     'B':[('D',2)],
     'C':[('G',3)],
     'D':[('G',1)],
     'G':[]}
h = {'A':4,'B':2,'C':2,'D':1,'G':0}
def astar(s,g1):  1 usage
    pq=[(0,s,0,[s])]
    while pq:
        f,n,c,p=heapq.heappop(pq)
        if n==g1: return p,c
        for nb,w in g[n]:
            heapq.heappush(pq, item: (c+w+h[nb],nb,c+w,p+[nb]))
def bfs(s,g1):  1 usage
    q=deque([(s,[s])])
    while q:
        n,p=q.popleft()
        if n==g1: return p
        for nb,w in g[n]:
            q.append((nb,p+[nb]))
p1,c=astar( s: 'A', g1: 'G')
p2=bfs( s: 'A', g1: 'G')
print("A*:", *p1, "Cost:", c)
print("BFS:", *p2)
```

```
A*: A B D G Cost: 4
BFS: A C G

Process finished with exit code 0
```

CONCLUSION: Hence we understand the concept of A* algorithm also we use A* Algorithm to find out goal state (Shortest path cost) in the given Graph.

DISCUSSION QUESTIONS:

Q.1) What do you mean by Search Algorithm?

Q.2) What are the differences between informed and uninformed search strategies?

Q.3) What is A* algorithm?

Q.4) What are the pros and cons of A* algorithm?

REFERENCES:

- https://www.javatpoint.com/ai-informed-search-algorithms
- https://www.geeksforgeeks.org/a-search-algorithm/
- https://www.educative.io/answers/what-is-the-a-star-algorithm